

Set Up Python Development Environment and Pipelines

Department of Statistics

Oregon State University

04/29/2026

Outline

01

Python environments

venv, Mamba(Conda), install
pytorch w/wo CUDA wheels

02

VS Code for DL

Remote-SSH into a GPU node, set
breakpoints for debugging, and
watch GPU utilization.

03

PyTorch Lightning

Modularize data, models, and
trainer logic for research prototype.

Python Environment

venv, Mamba(Conda), install pytorch w/wo CUDA wheels

Why using isolated environments?

- In isolated environments, you can install packages independently of your system-wide Python installation.
- Different projects use different versions of libraries without conflicts.
- Reproducibility: a teammate can install the exact environment from a file.

The rule of thumb

One environment per project.

venv

Virtual environment (venv) is a built-in Python module that creates isolated environments. Lightweight, no extra installation.

```
# Create an environment in ./venv
python3 -m venv .venv
#python3.11 -m venv .venv

# Activate
source .venv/bin/activate          # macOS / Linux

# Install + freeze
pip install --upgrade pip
pip install numpy torch torchvision
pip freeze > requirements.txt

# Reproduce on another machine
pip install -r requirements.txt    # inside a new activated venv
```

Tip: keep .venv in .gitignore and commit only requirements.txt.

Inside .venv

- Each env is a folder of symlinks/scripts pointing at a Python interpreter.

```
.venv/  
├── bin/                (Scripts/ on Windows)  
│   ├── python         ← symlink to your real python3  
│   ├── python3        ← symlink to your real python3  
│   ├── pip            ← small script  
│   └── activate        ← shell script that tweaks $PATH  
├── lib/  
│   └── python3.11/  
│       └── site-packages/ ← THIS is where pip installs go  
└── pyvenv.cfg          ← tiny config: "use that python over there"
```

- If something gets wrong, you can delete .venv/, recreate it from requirements.txt

Mamba

Mamba is a fast, drop-in CLI for the conda packaging ecosystem. Install packages via Miniforge to get the free conda-forge channel — no Anaconda commercial license.

```
# Install Miniforge (one-time):  
# github.com/conda-forge/miniforge  
  
# Create + activate  
mamba create -n dl python=3.11 -y  
mamba activate dl  
  
# Install PyTorch (CUDA 12.1)  
mamba install pytorch torchvision pytorch-cuda=12.1 -c pytorch -c nvidia  
  
# Export / recreate  
mamba env export > environment.yml  
mamba env create -f environment.yml
```

A mamba env can include gcc, g++, the CUDA toolkit, make, and assorted C/C++ libraries, all bundled inside the env.

venv vs Mamba

Aspect	venv + pip	Mamba
Manages Python interpreter	Uses your system Python	Installs its own Python
Manages non-Python libs	No (relies on OS / wheels)	Yes (CUDA, HDF5, MKL, FFmpeg, ...)
Disk footprint	Small — a few MB plus wheels	Large — full toolchains
Reproducibility file	requirements.txt / pyproject.toml	environment.yml
Best when	Pure Python / standard DL stack	Cluster, mixed C deps, no root

Installing PyTorch

- Two versions of PyTorch: a leaner version that only supports CPU computing and a version that supports both CPU and GPU computing.
- If your system supports a CUDA-compatible GPU, you can install the PyTorch version that supports GPU acceleration via CUDA.
- <https://pytorch.org/>

Installing PyTorch — CPU wheel

PyTorch ships pre-built wheels per (OS, Python, CUDA) tuple. The CPU-only wheel is light-weighted, good for laptops.

```
# CPU-only wheel (works on macOS, Linux, Windows)
pip install torch torchvision torchaudio \
  --index-url https://download.pytorch.org/whl/cpu
```

Verify

```
python -c "import torch; print(torch.__version__, torch.cuda.is_available())"
# 2.x.x False
```

Installing PyTorch — CUDA wheel

Check: (1) NVIDIA driver, (2) CUDA wheel index, (3) PyTorch version. The wheel ships its own CUDA runtime, i.e. you don't need to install the CUDA toolkit separately.

```
# Check the driver supports your CUDA wheel
nvidia-smi          # look at "CUDA Version: 12.x" (top right)

# Pick the matching index – examples:
#   cu121 → CUDA 12.1   cu118 → CUDA 11.8 (only needed for older version of CUDA)
pip install torch torchvision torchaudio \
  --index-url https://download.pytorch.org/whl/cu121
```

Sanity check on a GPU node

```
python -c "import torch; print(torch.cuda.is_available(), torch.cuda.get_device_name(0))"
# True  NVIDIA A100-SXM4-40GB
```

Environment Files

requirements.txt · pip + venv

```
# Install: pip install -r requirements.txt

--index-url https://download.pytorch.org/whl/cu121

torch==2.3.1
torchvision==0.18.1

numpy==1.26.4
pandas==2.2.2
matplotlib==3.8.4

lightning==2.3.0
torchmetrics==1.4.0

# dev tools
pytest==8.2.1
ruff==0.4.7
```

environment.yml · mamba

```
# Create: mamba env create -f environment.yml

name: dl
channels:
  - pytorch
  - nvidia
  - conda-forge
dependencies:
  - python=3.11
  - pytorch=2.3.1
  - pytorch-cuda=12.1
  - torchvision=0.18.1
  - numpy=1.26.4
  - pandas=2.2.2
  - lightning=2.3.0
  - torchmetrics=1.4.0
  - pip:
    - some-pure-py==1.0
```

environment.yml also captures channels and the Python version itself; requirements.txt assumes you've already activated the right interpreter.
write down the exact version of every package your project depends on, not just the names.

VS Code for DL development

Remote-SSH, interactive debugging, GPU monitoring

Why VS Code for deep learning

Python + Jupyter

Run notebooks as plain .py with cell markers, set breakpoint for debugging, inline DataFrame/Tensor viewer.

Remote-SSH built in

Develop directly on a GPU node. Files, terminal, debugger are all remote, only the UI is local.

Rich extension ecosystem

Pylance, Black, Ruff, Jupyter, Docker, GitLens, GPU monitors, LLM assistants.

Free & cross-platform

Works the same on macOS, Windows, Linux. Settings sync across machines via your account.

Setting up a new project

1

Initialize the project and venv

```
mkdir my-project && cd my-project
git init
python3 -m venv venv
echo "venv/" >> .gitignore
```

2

Open in VS Code & pick the interpreter

```
# File ▶ Open Folder...
# Cmd/Ctrl-Shift-P → "Python: Select Interpreter" → ./venv/bin/python
```

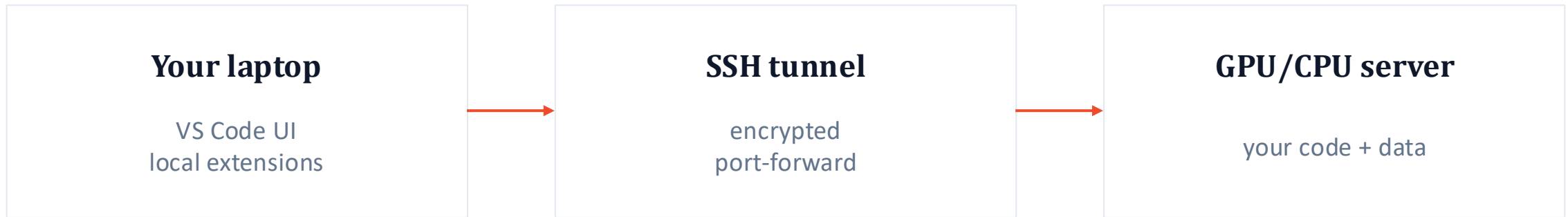
Once selected, the interpreter sticks per-project.

3

Install dependencies

```
# Integrated terminal (Ctrl+`) auto-activates the venv
pip install --upgrade pip
pip install -r requirements.txt
```

Remote-SSH



How it works

- VS Code installs a small server on the remote node on first connect.
- All extensions, terminals, debuggers, and file edits run remotely, and your laptop just renders the UI.
- Open a folder on the node through UI; integrated terminal is a real shell on the GPU host.

Setup Remote-SSH

1

Configure ~/.ssh/config

```
# Generate SSH key and copy to Novus cluster first
Host gpu001
    HostName gpu001
    ProxyJump novus.dri.oregonstate.edu
    User      yourname
    IdentityFile ~/.ssh/ed25519_1

Host novus.dri.oregonstate.edu
    HostName novus.dri.oregonstate.edu
    User      yourname
    IdentityFile ~/.ssh/ed25519_1
```

2

Install the extension

```
# Extensions panel → search "Remote - SSH"
```

3

Start an interactive session on cluster

```
srun -p gpu.q --gres=gpu:1 --pty zsh
```

4

Connect

```
# Cmd/Ctrl-Shift-P → "Remote-SSH: Connect to Host..." → gpu-node
# A new VS Code window opens, attached to the node.
# Open Folder → /home/yourname/projects/your-repo
```

Interactive debugging

Set a breakpoint inside `training_step`. Inspect tensors, step through batches, jump frames.

Minimal `.vscode/launch.json`

```
{
  "version": "0.2.0",
  "configurations": [{
    "name": "Train",
    "type": "debugpy",
    "request": "launch",
    "program": "src/train.py",
    "args": ["--config", "configs/base.yaml"],
    "console": "integratedTerminal",
    "justMyCode": false,
    "env": {"CUDA_VISIBLE_DEVICES": "0"}
  }]
}
```

Useful tricks

- Conditional breakpoints: trigger only when `loss > threshold`.
- Logpoints: log expressions without editing your code.
- Debug Console: evaluate expression inside any frame.
- `justMyCode: false` to step into Lightning / PyTorch internals.

Watching the GPU during training

From the terminal

```
# Plain  
nvidia-smi -l 1  
  
# Pretty TUI (recommended)  
pip install nvidia-top  
nvidia-top
```

nvidia-top shows per-process VRAM, utilization, and temperature in a colored TUI.

From inside VS Code

- GPU Monitor (extension) — sidebar widget with utilization and memory.
- Resource Monitor — CPU/RAM alongside GPU stats in the status bar.
- Pair with W&B for longer-running experiment views.

PyTorch Lightning

LightningModule · DataModule · Trainer

Raw PyTorch vs Lightning

Raw PyTorch loop

```
for epoch in range(num_epochs):
    model.train()
    for batch in train_loader:
        x, y = [t.to(device) for t in batch]
        optimizer.zero_grad()
        out = model(x)
        loss = loss_fn(out, y)
        loss.backward()
        optimizer.step()
    # plus: validation loop, AMP,
    # gradient clipping, checkpoints,
    # multi-GPU, logging, schedulers...
```

Lightning equivalent

```
class LitModel(L.LightningModule):
    def training_step(self, batch, idx):
        x, y = batch
        loss = loss_fn(self(x), y)
        self.log("train_loss", loss)
        return loss
    def configure_optimizers(self):
        return torch.optim.Adam(
            self.parameters(), lr=1e-3)

trainer = L.Trainer(max_epochs=10,
                    accelerator="gpu", precision="bf16-mixed")
trainer.fit(LitModel(), train_loader, val_loader)
```

LightningModule

```
import lightning as L
import torch, torch.nn.functional as F

class LitClassifier(L.LightningModule):
    def __init__(self, num_classes, lr=1e-3):
        super().__init__()
        self.save_hyperparameters()          # logged + checkpointed
        self.net = build_resnet(num_classes)
    def forward(self, x): return self.net(x)  # same as nn.Module
    def _step(self, batch, stage):
        x, y = batch
        logits = self(x)
        loss = F.cross_entropy(logits, y)
        acc = (logits.argmax(-1) == y).float().mean()
        self.log_dict({f"{stage}_loss": loss, f"{stage}_acc": acc}, prog_bar=True)
        return loss
    def training_step (self, b, i): return self._step(b, "train")
    def validation_step(self, b, i): return self._step(b, "val")
    def test_step      (self, b, i): return self._step(b, "test")
    def configure_optimizers(self):
        optimizer = torch.optim.AdamW(self.parameters(), lr=self.hparams.lr)
        scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=50)
        return {"optimizer": optimizer, "lr_scheduler": scheduler}
```

LightningDataModule

```
class CIFAR10DataModule(L.LightningDataModule):
    def __init__(self, data_dir="./data", batch_size=128, num_workers=4):
        super().__init__()
        self.save_hyperparameters()
    def prepare_data(self):                # called once, on rank 0
        CIFAR10(self.hparams.data_dir, train=True, download=True)
        CIFAR10(self.hparams.data_dir, train=False, download=True)
    def setup(self, stage):                # called on every process
        tfm = T.Compose([T.ToTensor(), T.Normalize(MEAN, STD)])
        if stage in ("fit", None):
            full = CIFAR10(self.hparams.data_dir, train=True, transform=tfm)
            self.train_ds, self.val_ds = random_split(full, [45000, 5000])
        if stage in ("test", None):
            self.test_ds = CIFAR10(self.hparams.data_dir, train=False, transform=tfm)
    def _loader(self, ds, shuffle):
        return DataLoader(ds, batch_size=self.hparams.batch_size,
                           shuffle=shuffle, num_workers=self.hparams.num_workers)
    def train_dataloader(self): return self._loader(self.train_ds, shuffle=True)
    def val_dataloader(self): return self._loader(self.val_ds, shuffle=False)
    def test_dataloader(self): return self._loader(self.test_ds, shuffle=False)
```

Trainer

```
from lightning import Trainer
from lightning.pytorch.callbacks import (
    EarlyStopping, ModelCheckpoint)
from lightning.pytorch.loggers import TensorBoardLogger

trainer = Trainer(
    max_epochs      = 50,
    accelerator     = "gpu",
    devices         = 2,          # 2 GPUs
    strategy        = "ddp",
    precision       = "bf16-mixed",
    gradient_clip_val = 1.0,
    log_every_n_steps = 25,
    callbacks = [
        EarlyStopping(monitor="val_loss", patience=5),
        ModelCheckpoint(monitor="val_acc", mode="max",
                        save_top_k=3)],
    logger = TensorBoardLogger("logs/", name="cifar10"))

trainer.fit (model, datamodule=dm)
trainer.test(model, datamodule=dm, ckpt_path="best")
```

What Trainer handles for you

- Device placement & mixed precision (fp16 / bf16)
- Multi-GPU / multi-node via strategy="ddp"
- Gradient accumulation, clipping, learning rate schedulers
- Checkpointing, resume, early stopping
- Pluggable loggers (TB, W&B, CSV, MLflow)
- fast_dev_run / overfit_batches sanity modes

How to modularize a Lightning project

```
my_project/
├─ configs/
│   ├─ base.yaml
│   └─ resnet50.yaml
├─ src/
│   ├─ data/
│   │   └─ cifar10.py          # LightningDataModule
│   ├─ models/
│   │   ├─ resnet.py           # nn.Module
│   │   └─ lit_classifier.py    # LightningModule
│   └─ callbacks/
│       └─ train.py            # builds Trainer + .fit()
├─ scripts/
│   └─ submit_slurm.sh
├─ requirements.txt
└─ pyproject.toml
```

Three layers, three responsibilities

Data

DataModule = downloads, splits, transforms, loaders.

Model

nn.Module = neural network architecture.
LightningModule = step + optimizer + logging.

Training

Trainer = devices, callbacks, logger.
Specify hyperparameters by a config so runs are reproducible.

Summary

- Python environment
- Install Pytorch
- VS code IDE
- Lightning modules: LightningModule + DataModule + Trainer